Name: Umesh Dhakal

Course: MSCS634

Professor: Dr. Satish Penmatsa

March 27,2026

Assignment 3

1) Data Warehouse and OLAP Operations

A. Snowflake Schema Design

Fact table

Fact_Enrollment: student_key (FK), course_key (FK), term_key (FK), faculty_key (FK), total_enrollment, avg_score.

Dimension Tables

1. Student Dimension

Dim_Student: student_key (PK), student_id , student_name, gender, age, department_key (FK).

Dim_Student_Department: department_key (PK), department_name, school_key (FK).

Dim_Student_School: school_key (PK), school_name.

Student hierarchy: student → department → school → all

2. Course Dimension

Dim_Course: course_key (PK), course_id, course_name, credits, subject_key (FK).

Dim_Course_Subject: subject_key (PK), subject_name, program_key (FK).

Dim_Course_Program: program_key (PK), program_name.

Course hierarchy: course → subject → program → all

3. Term Dimension

Dim_Term: term_key (PK), term_name, semester_key (FK).

Dim_Semester: semester_key (PK), semester_name, academic_year_key (FK).

Dim_Academic_Year: academic_year_key (PK), academic_year.

Term hierarchy: term → semester → academic year → all

4. Faculty Dimension

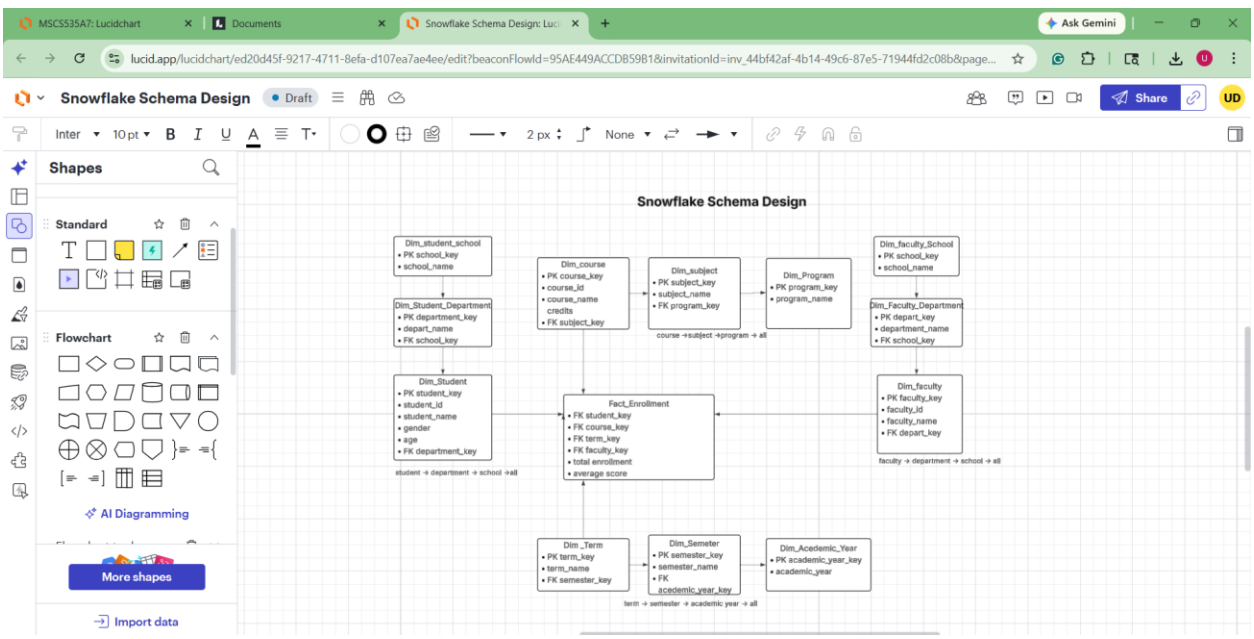
Dim_Faculty: faculty_key (PK), faculty_id, faculty_name, department_key (FK).

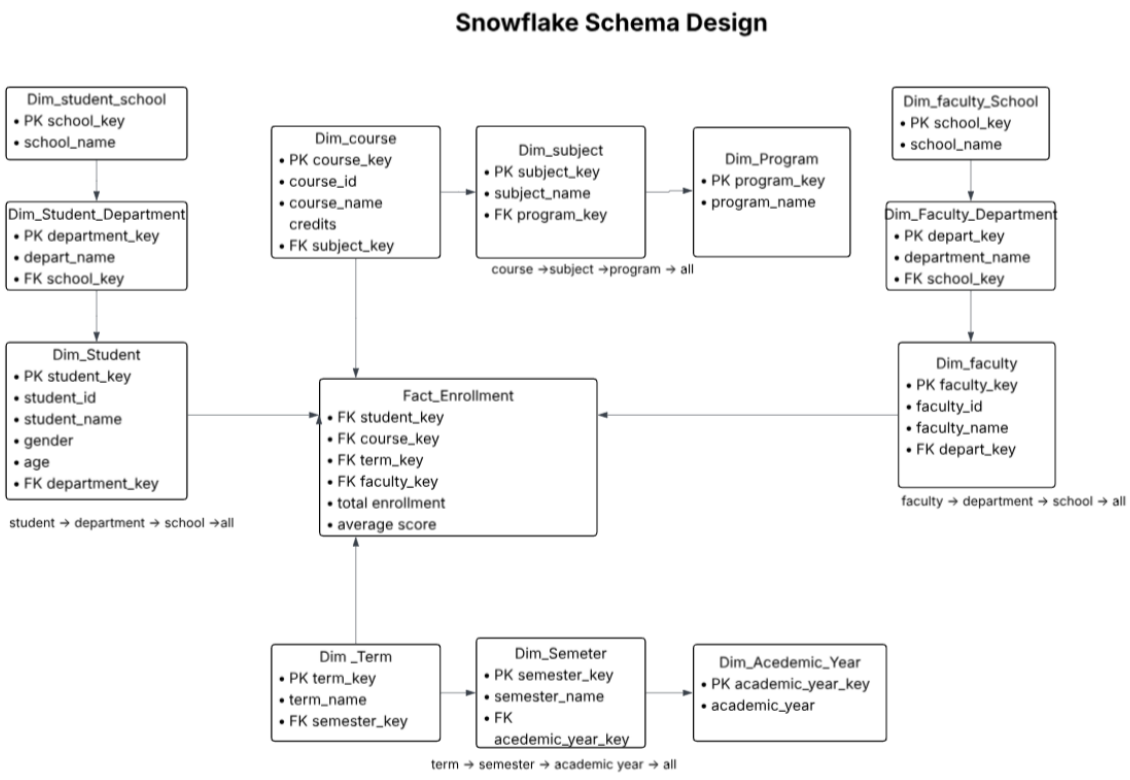
Dim_Faculty_Department: department_key (PK), department_name, school_key (FK).

Dim_Faculty_School: school_key (PK), school_name.

Faculty hierarchy: faculty → department → school → all

Snowflake Schema Design





B. We start with the base cuboid [student, course, term, faculty]. To compute the average score for each Data Mining course taken by Global Tech Institute students, I would use these OLAP operations.

- First, I would use slice on the course dimension to keep only the Data Mining course or Data Mining related courses.
- Then, I would use roll-up on the student dimension from student level to all students. This is because I do not need one score for each student. I need the average score for the course.
- Next, I would use roll-up on the term dimension from term level to all terms if I want the overall average across all terms.
- After that, I would use roll-up on the faculty dimension from faculty level to all faculty, so the result is not separated by instructor.

So, the result becomes close to: [all students, Data Mining course, all terms, all faculty]

At this level, the avg_score measure gives the average score for each Data Mining course. So, the OLAP steps are slice on course = Data Mining, roll-up student to all, roll-up term to all, roll-up faculty to all.

C. Each dimension has 4 levels, including all levels.

Student = 4 levels, Course = 4 levels, Term = 4 levels, Faculty = 4 levels. So, the total number of cuboids = student levels * course levels * term levels * faculty levels

= 4 * 4 * 4 * 4 = 256

So, the data cube contains 256 cuboids in total. This includes the base cuboid, the apex cuboid, and all the cuboids in between.

2) Data Warehouse and OLAP Operations

A. Star schema

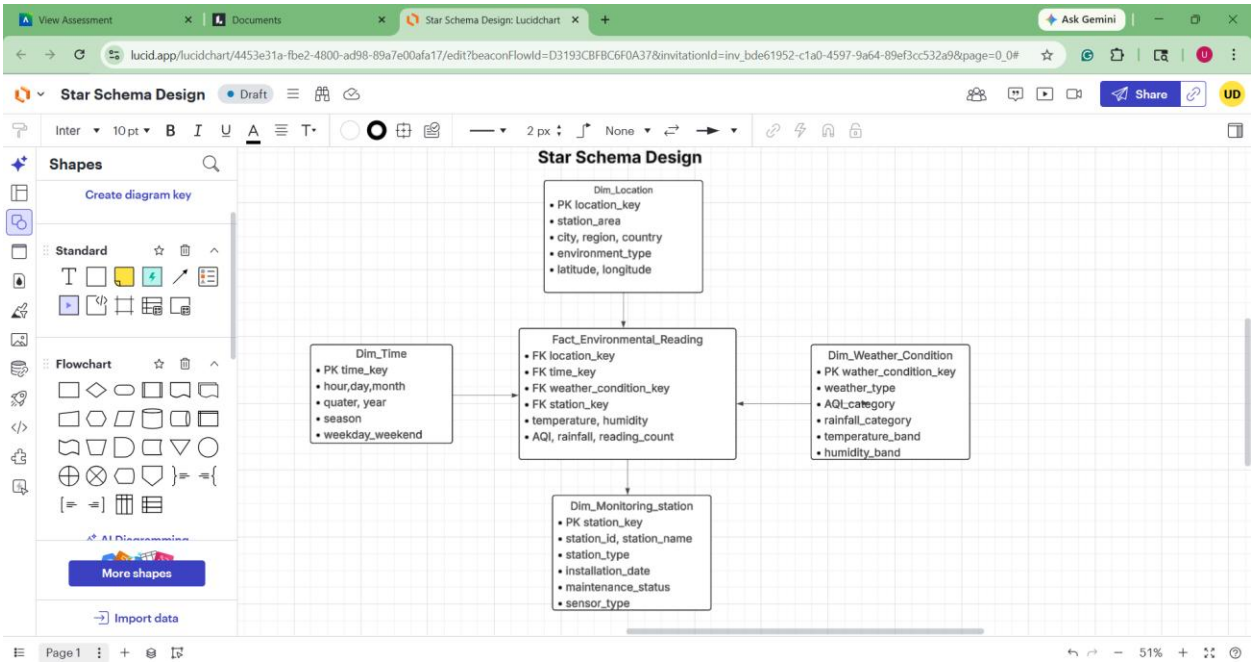
Fact table

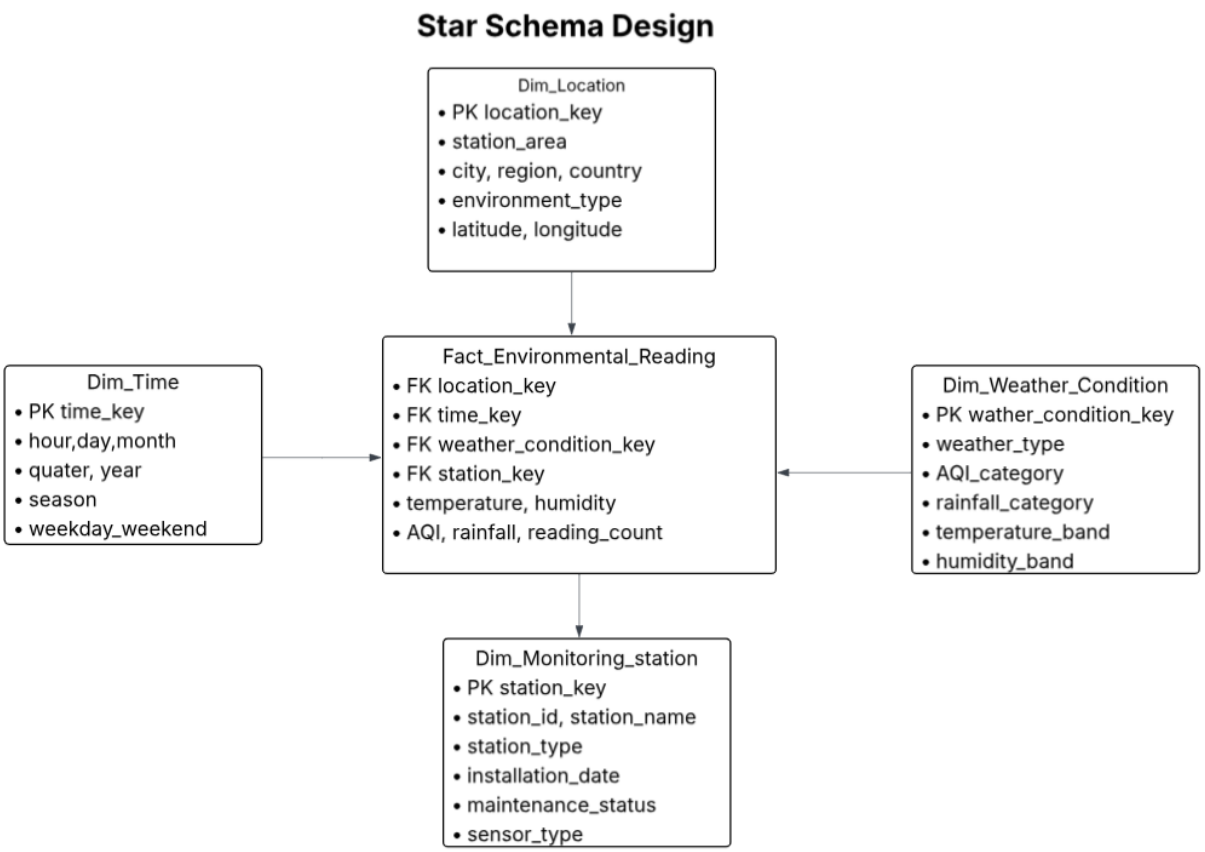
Fact_Environmental_Reading: location_key (FK), time_key (FK), weather_condition_key (FK), station_key (FK), temperature, humidity, AQI, rainfall, reading_count.

Dimension tables

Dim_Location: location_key (PK), station_area, city, region, country, environment_type, latitude, longitude.
Dim_Time: time_key (PK), hour, day, month, quarter, year, season, weekday_weekend
Dim_Weather_Condition: weather_condition_key (PK), weather_type, AQI_category, rainfall_category, temperature_band, humidity_band.
Dim_Monitoring_Station: station_key (PK), station_id, station_name, station_type, installation_date, maintenance_status, sensor_type.

A. Star schema


View Assessment
Documents
Star Schema Design: Lucidchart
Ask Gemini
lucid.app/lucidchart/4453e31a-fbe2-4800-ad98-89a7e00afa17/edit?beaconFlowId=D3193CBFBC6F0A37&invitationId=inv_bde61952-c1a0-4597-9a64-89ef3cc532a9&page=0_0#
Star Schema Design
Draft
Share
UD
Shapes
Create diagram key
Standard
Flowchart
More shapes
Import data
Star Schema Design
Dim_Location
• PK location_key
• station_area
• city, region, country
• environment_type
• latitude, longitude
Dim_Time
• PK time_key
• hour,day,month
• quater, year
• season
• weekday_weekend
Fact_Environmental_Reading
• FK location_key
• FK time_key
• FK weather_condition_key
• FK station_key
• temperature, humidity
• AQI, rainfall, reading_count
Dim_Weather_Condition
• PK wather_condition_key
• weather_type
• AQI_category
• rainfall_category
• temperature_band
• humidity_band
Dim_Monitoring_station
• PK station_key
• station_id, station_name
• station_type
• installation_date
• maintenance_status
• sensor_type
Page 1
51%


Star Schema Design
Dim_Location
• PK location_key
• station_area
• city, region, country
• environment_type
• latitude, longitude
Dim_Time
• PK time_key
• hour,day,month
• quater, year
• season
• weekday_weekend
Fact_Environmental_Reading
• FK location_key
• FK time_key
• FK weather_condition_key
• FK station_key
• temperature, humidity
• AQI, rainfall, reading_count
Dim_Weather_Condition
• PK wather_condition_key
• weather_type
• AQI_category
• rainfall_category
• temperature_band
• humidity_band
Dim_Monitoring_station
• PK station_key
• station_id, station_name
• station_type
• installation_date
• maintenance_status
• sensor_type

B. Here, we start from the base cuboid [location, time, weather condition, monitoring station]. This is the most detailed level because it shows data for each location, each time, each weather condition, and each monitoring station. To analyze the average temperature trends for each city across the years, I would use these OLAP operations:

- First, I would roll-up the time dimension from hour level to year level.
- Next, I would roll-up the location dimension from detailed location to city.
- Then, I would roll-up the monitoring station dimension to all stations so the result is not separated by station.
- After that, I would roll-up the weather condition dimension to all weather conditions if I want the overall temperature trend.
- After all these steps, the result becomes close to [city, year, all weather conditions, all stations]. At that level, the average temperature measure shows the temperature trend for each city across the years.

So, the main OLAP operations used here are roll-up on time, location, monitoring station, and weather conditions. If needed, slice or dice can also be used to focus on a specific city, year, or weather condition.

C. Each dimension has 4 levels, including all levels.

Location = 4 levels
Time = 4 levels
Weather condition = 4 levels
Monitoring station = 4 levels
The total number of cuboids = Location * Time * Weather Condition * Monitoring Station

$$= 4 * 4 * 4 * 4 = 256$$

So, the data cube contains 256 cuboids in total. This includes the base cuboid, the apex cuboid, and all the cuboids in between.

Constraint-Based Pattern Mining in Retail Data

A. This is a monotone constraint. The reason is that if a pattern already contains at least one electronics item, then any larger pattern made by adding more items will still contain at least one electronics item. So, once the condition is true, it stays true for all supersets. This kind of constraint can be handled by focusing only on item sets that include at least one electronics product. During mining, patterns that do not contain any electronics item can be ignored. This helps reduce unnecessary patterns and makes the mining process faster.

B. This is also a monotone constraint. Since all item prices are non-negative, if the total price of one pattern is already greater than $300, then adding more items will keep the total price above $300 or make it even larger. So, this property also stays true for supersets. This constraint can be handled by checking the total price of the pattern during mining. Once a pattern satisfies the condition, its larger supersets will also satisfy it. This helps in reducing repeated checking and improves efficiency.

C. This case has two constraints together. The first part, which says the pattern must include one discounted item, is a monotone constraint because once a discounted item is included, adding more items does not remove it. The second part, which says the total combined price must be less than $100, is an anti-monotone constraint because if a pattern already goes above $100, then adding more items will only increase the total more, so its supersets will also fail. This can be handled by first keeping only patterns that include at least one discounted item and then pruning patterns as soon as the total price becomes $100 or more. This is effective because many useless candidates can be removed early.

D. This is a convertible constraint. The reason is that average price is not directly monotone or anti-monotone. When new items are added, the average can increase or decrease, so it is harder to prune patterns directly using the average alone. This type of constraint can be handled by using item ordering or transformation methods so that it behaves like a monotone or anti-monotone constraint during mining. Another way is to use upper and lower bounds to check whether a pattern can still satisfy the average price range. This makes the mining process more efficient.

4) Mining Representative Patterns in Large Datasets

A. To select the top k most representative patterns, I would first mine the frequent patterns by using a minimum support threshold. This helps remove very weak patterns and keeps only the ones that appear enough times in the dataset. After that, I would give each pattern a simple score based on support and pattern length, because patterns that appear more often and contain more useful items are usually more important. Then I would sort the patterns from highest score to lowest score and select them one by one. Before keeping a new pattern, I would also compare it with the patterns already selected. If it is too similar, I would skip it and move to the next one. This helps reduce redundancy and keeps the final patterns more diverse and meaningful. In a real-life retail setting, this would be useful because a store manager does not need a very long list of almost identical patterns. For example, if many customers buy a laptop with a mouse, a keyboard, a headset, a charger, or a laptop bag, the system should not keep showing many patterns that all mean almost the same thing. Instead, it should keep the strongest and most different patterns so that managers can use them for bundle offers, promotions, inventory planning, and product placement.

B. Let suppose the transactions are:

T1 = {Laptop, Mouse, Keyboard, Headset}

T2 = {Laptop, Mouse, Docking Station}

T3 = {Chromebook, Mouse, Keyboard, Charger}

T4 = {Laptop, Monitor, Mouse}

T5 = {Chromebook, Keyboard, Case, Charger}

T6 = {Laptop, Keyboard, Mouse, Laptop Bag}

T7 = {Chromebook, Mouse, Case}

T8 = {Monitor, Keyboard, Mouse}

If the minimum support is 2, some frequent patterns are {Laptop, Mouse} = 4, {Mouse, Keyboard} = 4, {Chromebook, Charger} = 2, {Chromebook, Case} = 2, and {Monitor, Mouse} = 2. If k = 4, I would select {Laptop, Mouse}, {Mouse, Keyboard}, {Chromebook, Case}, and {Monitor, Mouse}. I selected these because they have reasonable support and they are not all showing the exact same buying pattern. For example, Laptop, Mouse} represents a common computer accessory pattern, {Mouse, Keyboard} shows a general accessory combination, {Chromebook, Case} shows a different type of need related to protection and portability, and {Monitor, Mouse} reflects another separate buying pattern. This makes them more representative and more useful than keeping many repeated patterns that overlap too much.

C. This method is effective because it keeps patterns that are important and also different from each other. Support helps keep strong patterns, and the similarity check helps remove redundant ones. So, the final result is easier to understand and gives a better summary of the data. This is useful in real life because a retail store or an online electronics store may have thousands of transactions, and managers need a smaller set of patterns that they can actually use. For example,

if the final selected patterns show that laptop buyers often also buy a mouse, a Chromebook is often bought with a case, or a monitor is often bought with a mouse, the store can use that information for shelf arrangement, combo discounts, or recommendation systems. One limitation is that some rare and interesting patterns may not be selected if their support is low. Also, if only a small number of patterns is kept, some details may be left out. Even with that, this method is still useful because it reduces redundancy, keeps the most meaningful patterns, and gives a clearer view of customer buying behavior.